

Software Requirements and Design Document

for

LOCUS ANALYTICS

Real-Estate Analytics Platform for Pakistan

Prepared by

Fasih Ul Mubashir (24i-0517) — System Engineer
Arham Manzoor (24i-0640) — Lead Data Scientist
Ayaan Aman (24i-0663) — Frontend Architect

SCOPE: JavaFX desktop application covering FMV estimation, rental yield, ROI, price trends, heatmaps, investment clusters, and PDF reporting for the Karachi, Lahore, and Islamabad markets.

DATE: May 7, 2026

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Product Scope
 - 1.3 Title
 - 1.4 Objectives
 - 1.5 Problem Statement
- 2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 List of Use Cases
 - 2.4 Extended Use Cases
 - 2.5 Use Case Diagram
- 3. Other Nonfunctional Requirements
 - 3.1 Performance Requirements
 - 3.2 Safety Requirements
 - 3.3 Security Requirements
 - 3.4 Software Quality Attributes
 - 3.5 Business Rules
 - 3.6 Operating Environment
 - 3.7 User Interfaces
- 4. Domain Model
- 5. System Sequence Diagram
- 6. Sequence Diagram
- 7. Class Diagram
- 8. Package Diagram
- 9. Deployment Diagram

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the requirements and design of **LOCUS Analytics v1.0**, a JavaFX-based desktop platform for real-estate analytics in Pakistan. The document covers the entire system: the JavaFX presentation layer, the service and DAO layers, the PostgreSQL persistence layer, the Python ETL pipeline, and the Ridge-regression ML pricing model. It is intended for the development team, evaluators, and future maintainers.

1.2 Product Scope

LOCUS Analytics enables property analysts and system administrators to ingest listings from public sources, estimate fair market value, calculate rental yield and ROI, visualise price trends and density heatmaps, identify investment clusters via K-Means, and produce branded PDF valuation reports. The product is delivered as a single Maven-built fat JAR backed by a PostgreSQL 15 database and supporting Python utilities (ETL, seeding, model retraining). The benefits are: (i) data-driven valuations replacing anecdotal pricing, (ii) a unified analyst workstation for the Karachi/Lahore/Islamabad markets, and (iii) automated, auditable PDF deliverables for clients.

1.3 Title

LOCUS Analytics — A Desktop Real-Estate Intelligence and Valuation Platform for Pakistan.

The aim is to provide property analysts an integrated workbench for valuation, yield analysis, and investment-cluster discovery using a transparent, retrainable ML model.

1.4 Objectives

- Estimate Fair Market Value (FMV) of any residential property using a Ridge-regression model trained on locality, area, bedrooms, bathrooms, property type, and amenities.
- Compute rental yield and ROI for any listing or hypothetical investment scenario.
- Visualise locality-level price trends across configurable time ranges.
- Render a Google-Maps heatmap of listing density and price intensity.
- Discover investment clusters using K-Means over yield, growth, and price-per-sqft.
- Generate professional, branded PDF valuation reports for clients.
- Run an ETL pipeline that scrapes/synthesises listings, deduplicates, and loads into PostgreSQL.
- Allow administrators to manage users, listings, and ML/ETL system configuration.

1.5 Problem Statement

Real-estate decisions in Pakistan are dominated by anecdotal pricing, fragmented portals, and a lack of analytical tooling. Buyers, sellers, and analysts rarely have access to consistent fair-market valuations or rental-yield/ROI calculators tied to a verifiable dataset. Existing portals show listings but not analytics; spreadsheets used by analysts are error-prone and non-reproducible.

LOCUS Analytics addresses this by combining a curated PostgreSQL dataset, a transparent ML pricing model, and an analyst-grade JavaFX UI. Office automation replaces the current manual valuation workflow: a report that previously required several hours of spreadsheet work is generated as a branded PDF in seconds. **Feasibility:** the technology stack (Java 17, JavaFX, PostgreSQL, Python, scikit-learn) is open-source; the team has the required Java, Python, and DBMS skills; the dataset is bootstrapped via a deterministic seed script (800 realistic properties) and extended via the ETL pipeline, so no third-party data licence is needed for the prototype.

2. Overall Description

2.1 Product Perspective

LOCUS Analytics is a new, self-contained product — not a follow-on of any prior system. It is composed of three cooperating subsystems that share a single PostgreSQL database:

- **JavaFX Desktop Client** — the analyst-facing application (login, 14 screens, controllers, services).
- **Python ETL Pipeline** — extracts/synthesises listings, cleans them, and loads them into the *property* table while emitting progress to the *etl_job* table.
- **Python ML Module** — trains/retrains the Ridge regression model and serialises it to *ml/model.json*, which the Java *LinearRegressionPredictor* hot-reloads.

External interfaces: (i) Google Maps JavaScript API for the heatmap WebView, (ii) PostgreSQL JDBC, (iii) the local filesystem for the generated *valuation-report-*.pdf* artefacts and the *backups/* directory.

2.2 Product Functions

At a high level, the product offers:

- Authentication with role-based access (analyst vs. admin), BCrypt password hashing.
- FMV estimation backed by a hot-reloadable Ridge regression model.
- Rental-yield and ROI calculators with input validation.
- Property search with multi-criteria filters and pagination.
- Side-by-side property comparison.
- Price-trend charts at the locality/city level.
- Density and price heatmap rendered through a Google Maps WebView.
- K-Means investment-cluster discovery with configurable parameters.
- Branded PDF valuation report generation and persistence.
- ETL pipeline orchestration with live progress, deduplication, and resume support.
- Admin-only listing management, user management, and system configuration.

2.3 List of Use Cases

#	Use Case	Primary Actor	Screen
UC-1	Estimate Fair Market Value	Property Analyst	FMV Estimate

UC-2	Calculate Rental Yield	Property Analyst	Rental Yield
UC-3	Calculate ROI	Property Analyst	ROI Calculator
UC-4	Search Properties	Property Analyst	Search
UC-5	Compare Properties	Property Analyst	Compare
UC-6	View Price Trends	Property Analyst	Price Trends
UC-7	Render Property Heatmap	Property Analyst	Heatmap
UC-8	Generate Valuation Report (PDF)	Property Analyst	Reports
UC-9	Identify Investment Clusters	Property Analyst	Clusters
UC-10	Run ETL Pipeline	System Administrator	ETL Dashboard
UC-11	Manage Listings	System Administrator	Listings
UC-12	System Configuration	System Administrator	Config
UC-13	Authenticate User	All Users	Login

2.4 Extended Use Cases

UC-1: Estimate Fair Market Value (extended)

Primary Actor: Property Analyst **Stakeholders:** Client, System Admin

Preconditions: User is authenticated; *ml/model.json* exists and has been loaded.

Trigger: Analyst opens the FMV Estimate screen and submits a property profile.

Main Success Scenario:

- Analyst enters city, locality, area (sqft), bedrooms, bathrooms, type, and amenity flags.
- *InputValidator* rejects out-of-range values (e.g., area < 100 sqft).
- *ValuationServiceImpl* assembles a feature vector and calls *LinearRegressionPredictor*.
- Predictor returns the FMV in PKR; the service persists a *Valuation* row.
- Controller renders the FMV plus a confidence band based on locality variance.

Extensions: (a) Model file missing → service raises a friendly error and the UI shows a retry banner.
(b) Locality unknown to the model → fall back to city-level mean encoding.

UC-8: Generate Valuation Report (extended)

Primary Actor: Property Analyst

Preconditions: A *Valuation* record exists for the target property.

Main Success Scenario:

- Analyst selects a stored valuation and clicks *Generate PDF*.

- *ValuationReportServiceImpl* aggregates property facts, FMV, comparables, and trend data.
- *ReportPdfServiceImpl* builds a branded PDF (logo, charts, tables) into *generated-reports/*.
- A *ValuationReport* row is inserted with the file path and SHA-256 hash.
- The UI offers *Open* and *Reveal in folder* actions.

Extensions: (a) PDF write fails (disk full) → DAO transaction is rolled back; UI shows the I/O error.

UC-10: Run ETL Pipeline (extended)

Primary Actor: System Administrator

Preconditions: Admin is logged in; the database is reachable.

Main Success Scenario:

- Admin clicks *Run ETL*; *ETLServiceImpl* spawns the Python *run_etl.py* process.
- Python writes per-batch progress into the *etl_job* row created by the service.
- The JavaFX progress bar polls *ETLJobDAOImpl.findLatest()* and animates.
- On completion, the row is marked *SUCCESS* and the model can be retrained.

Extensions: (a) Crash mid-run → resume on next invocation skips already-loaded *url_hash* values.
 (b) Network/DB error → row marked *FAILED* with the captured stderr.

2.5 Use Case Diagram

The diagram below summarises actors and use cases. Two actors interact with the system: the **Property Analyst** (analytical use cases UC-1 to UC-9) and the **System Administrator** (operational use cases UC-10 to UC-12). Both inherit from the abstract **User** actor that performs UC-13 (Authenticate).

Actor	→	Use Cases
User (abstract)	→	UC-13 Authenticate
Property Analyst	→	UC-1 Estimate FMV, UC-2 Rental Yield, UC-3 ROI, UC-4 Search, UC-5 Compare, UC-6 Price Trend, UC-7 Forecast, UC-8 Generate Report
System Administrator	→	UC-10 Run ETL, UC-11 Manage Listings, UC-12 Configure System (plus all Analyst use cases)

«include» relationships: every analyst/admin use case includes UC-13 (Authenticate). «extend»:
 UC-8 (Generate Report) extends UC-1 when the analyst chooses to export an FMV result.

3. Other Nonfunctional Requirements

3.1 Performance Requirements

- FMV estimation must complete in < 200 ms on the analyst desktop after the model is loaded.
- Property search with up to four filters over 50,000 rows must return the first page in < 500 ms (supported by indexes on *city*, *locality*, and *price*).
- PDF report generation must complete in < 5 s for a single property with up to 10 comparables.
- ETL throughput target: ≥ 200 listings/second insert rate on the seeded dataset.
- Application cold-start (login screen visible) must be < 4 s on a 4-core / 8-GB machine.
- Heatmap initial render < 3 s for 5,000 points.

3.2 Safety Requirements

- All destructive admin actions (delete listing, purge ETL job) require an explicit confirmation dialog.
- Database backups are produced weekly via *scripts/backup.sh/bat* with 4-week retention; this protects against accidental data loss.
- PDF report generation never overwrites existing files — each report is named by UUID.
- The ML model is loaded into memory at startup and on file-change; a malformed model file falls back to the previously-loaded version rather than crashing the app.

3.3 Security Requirements

- Passwords are stored as BCrypt hashes (cost factor 10) in *app_user.password_hash*.
- Role-based access control: analyst vs. admin; admin-only screens are gated by *SceneManager*.
- All DAO writes go through PreparedStatements — no string-concatenated SQL.
- Every write operation emits an *audit_log* row (actor, action, target, timestamp) via a PostgreSQL trigger.
- Database credentials and the Google Maps API key live in *config.properties* and are excluded from version control.
- The Google Maps WebView is sandboxed and cannot navigate outside the configured map domain.

3.4 Software Quality Attributes

- **Maintainability:** strict layering (UI → Service → DAO → DB) with interface/impl separation; service stubs enable test isolation.
- **Testability:** 63 JUnit-5 unit tests across 7 service classes, runnable without a database.
- **Portability:** single fat JAR built with the Maven Shade plugin; runs on Windows, Linux, macOS with JDK 17+.
- **Reliability:** ETL is idempotent (deduplication via *url_hash*) and resumable after a crash.
- **Usability:** keyboard navigation, consistent CSS theming, and a fixed top navigation bar.
- **Reusability:** service interfaces are decoupled from JavaFX so they can be reused by a future REST/CLI front-end.

3.5 Business Rules

- **BR-1:** Only *admin* users may run the ETL pipeline or modify system configuration.
- **BR-2:** Only *admin* users may delete or hard-edit a property listing; analysts may create valuations but not mutate source listings.
- **BR-3:** A valuation older than 30 days is flagged as *STALE* on the report.
- **BR-4:** A listing with area < 100 sqft or price < PKR 100,000 is rejected by ETL cleaning.
- **BR-5:** Generated PDF reports are immutable; corrections require a new report.
- **BR-6:** Every authenticated session lasts at most 8 hours, then a re-login is required.

3.6 Operating Environment

- **OS:** Windows 10/11, Ubuntu 22.04 LTS, macOS 13+ (any host with a JVM 17 runtime).
- **JVM:** OpenJDK 17 or higher with JavaFX 21 modules.
- **Database:** PostgreSQL 15+ on the same host or a reachable network host (Oracle 19c also supported via *schema_oracle.sql*).
- **Python:** 3.10+ with *scikit-learn*, *pandas*, *numpy*, *psycpg2-binary*.
- **Hardware:** minimum 4-core CPU, 8 GB RAM, 1 GB free disk for backups and reports.
- **Network:** outbound HTTPS for Google Maps tiles only; the rest is local.

3.7 User Interfaces

The UI is a JavaFX application with 14 FXML screens following a fixed-navbar layout. The brand palette uses navy (#0B3D91) as primary, white as background, and accent green for positive metrics. Standard controls present on every screen: top navigation bar with the LOCUS logo, screen title, user badge, and **Help** + **Logout** buttons. Forms validate on blur; error messages appear in a red banner below the form. Long-running operations (ETL, PDF generation) display a determinate progress bar driven by *UiAnimationHelper*. Keyboard shortcuts: *Ctrl+L* logout, *Ctrl+R* refresh, *Esc* close modal. Detailed UI specifications live in *BrandAssets.java* and the FXML stylesheet.

4. Domain Model

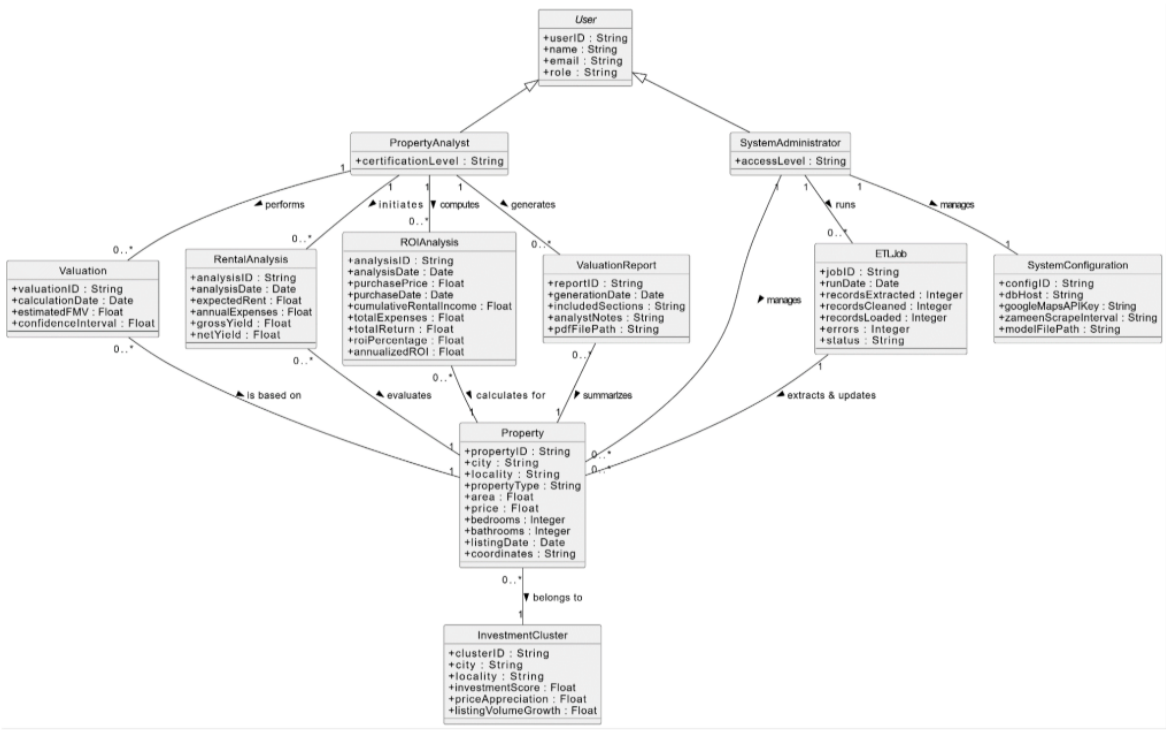
The domain model captures the conceptual entities of the property-analytics domain and their relationships, independent of any storage or UI concern. The principal concepts are:

Concept	Description	Key Attributes
User	Authenticated actor of the system.	userId, name, email, role, passwordHash
PropertyAnalyst	User who performs valuations and analyses.	(specialisation of User)
SystemAdministrator	User who runs ETL and manages config.	(specialisation of User)
Property	A single real-estate listing.	propertyId, city, locality, area, beds, baths, price, type
Valuation	An FMV estimate produced for a property.	valuationId, propertyId, fmv, generatedAt
RentalAnalysis	Yield computation for a property.	annualRent, grossYield, netYield
ROIAnalysis	ROI scenario computation.	investment, holdingYears, expectedROI
InvestmentCluster	K-Means cluster of localities/properties.	clusterId, centroid, members
ValuationReport	Generated PDF artefact.	reportId, valuationId, filePath, sha256
ETLJob	A run of the ETL pipeline.	jobId, startedAt, status, processed, failed
AuditLog	Append-only record of writes.	logId, actor, action, target, ts
SystemConfiguration	Tunable runtime parameters.	key, value, updatedBy

Key associations: *User* ◇— produces —◇ *Valuation*; *Property* 1—* *Valuation*; *Valuation* 1—1 *ValuationReport*; *Property* *—* *InvestmentCluster* (via cluster membership); *SystemAdministrator* 1—* *ETLJob*; every write produces exactly one *AuditLog* entry.

Figure 4.1 — Domain Model (rendered)

Domain Model:



5. System Sequence Diagram

The SSD below describes the black-box interaction between the **Property Analyst** actor and the **LOCUS Analytics System** for the central use case **UC-1: Estimate Fair Market Value**. Only system-boundary events are shown.

Step	Actor → System	System → Actor
1	login(email, password)	
2		sessionToken, role
3	openFmvScreen()	
4		screen ready
5	submitFmvRequest(city, locality, area, beds, baths, type, amenities)	
6		fmvAmount, confidenceBand, valuationId
7	requestPdfReport(valuationId)	
8		reportFilePath
9	logout()	
10		ack

Notation: each numbered row is one message crossing the system boundary. Steps 1–2, 5–6, 7–8, and 9–10 are synchronous request/response pairs; the system performs internal work (model lookup, DB write, PDF rendering) entirely as a black box.

6. Sequence Diagram

The white-box sequence diagram below expands UC-1 (Estimate FMV) into the actual collaboration between the JavaFX controller, service, predictor, and DAO classes.

#	Caller	Callee	Message
1	FMVEstimateController	InputValidator	validate(form)
2	InputValidator	FMVEstimateController	ok / ValidationException
3	FMVEstimateController	ValuationServiceImpl	estimateFMV(featureDto)
4	ValuationServiceImpl	LinearRegressionPredictor	predict(features)
5	LinearRegressionPredictor	ValuationServiceImpl	fmv (PKR)
6	ValuationServiceImpl	ValuationDAOImpl	save(Valuation)
7	ValuationDAOImpl	DBConnection	getConnection()
8	ValuationDAOImpl	PostgreSQL	INSERT INTO valuation ...
9	PostgreSQL	ValuationDAOImpl	valuationId
10	ValuationDAOImpl	AuditLogDAOImpl	log(actor, 'CREATE_VALUATION', valuationId)
11	ValuationServiceImpl	FMVEstimateController	ValuationResult{fmv, band, id}
12	FMVEstimateController	JavaFX Scene	render(result)

The model strictly traverses UI → Service → DAO → DB. Audit logging is invoked synchronously after each write, ensuring every mutation is recorded before the controller returns.

7. Class Diagram

The class diagram of LOCUS Analytics is organised around four logical groups: **model**, **dao**, **service**, and **ui.controller**. The table below summarises the principal classes; «*interface*» stereotypes are noted where applicable.

Layer	Class / Interface	Key Members
model	User, PropertyAnalyst, SystemAdministrator	userId, name, email, role; role-specific behaviour
model	Property	propertyId, city, locality, area, beds, baths, price, type
model	Valuation	valuationId, propertyId, fmv, confidence, generatedAt
model	ValuationReport	reportId, valuationId, filePath, sha256
model	RentalAnalysis, ROIAnalysis	yield, growth, ROI inputs/outputs
model	InvestmentCluster	clusterId, centroid, members[]
model	ETLJob, AuditLog, SystemConfiguration	operational metadata
model.dto	SearchFilter, PagedResult, TrendPoint, HeatmapPoint, ClusterParams, ROIInput, TimeRange, LocalityMetric, ComparisonMetric	
dao «interface»	PropertyDAO, ValuationDAO, ValuationReportDAO, RentalAnalysisDAO, ROIAnalysisDAO, InvestmentClusterDAO, ETLJobDAO, AuditLogDAO, SystemConfigurationDAO	
dao.impl	*DAOImpl (10 classes)	JDBC implementations using HikariCP
config	DBConnection	HikariCP DataSource, getConnection()
ml	LinearRegressionPredictor	loadModel(path), predict(features), hot-reload watcher
service «interface»	AuthenticationService, ValuationService, RentalYieldService, ROIService, SearchService, CompareService, PriceTrendService	
service.impl	*ServiceImpl + *ServiceStub (parallel hierarchy)	real impls and in-memory stubs
service.validation	InputValidator	form-level validation + ValidationException
ui	LocusAnalyticsApp, Main, SceneManager, ServiceRegistry, BrandAssets	
ui.controller	LoginController, MainController, ServiceAwareController, ValidationBridge	
ui.controller.screen	AbstractScreenController + 14 screen controllers	FMV, RentalYield, ROI, Search, Compare, PriceTrend, Heatmap, ReportPdf

Key relationships: (i) every **ServiceImpl* depends on one or more **DAO* interfaces; (ii) every **DAOImpl* depends on *DBConnection*; (iii) every screen controller extends *AbstractScreenController* and is wired through *ServiceRegistry*; (iv) *PropertyAnalyst* and *SystemAdministrator* generalise *User*; (v) *ValuationServiceImpl* aggregates *LinearRegressionPredictor*; (vi) *ValuationReportServiceImpl* composes *ReportPdfServiceImpl*.

8. Package Diagram

The Java source is organised under the root package **com.locus** and follows a strict downward dependency rule: an upper layer may only depend on the layers below it.

Package	Responsibility	Depends On
com.locus.ui	JavaFX bootstrap, scene management, brand assets.	ui.controller, service
com.locus.ui.controller	Shell controllers (Login, Main, Navigation).	ui.controller.screen, service
com.locus.ui.controller.screen	One controller per FXML screen (14 screens).	service, model.dto
com.locus.service	Service interfaces — the domain API.	model, model.dto
com.locus.service.impl	Production + stub implementations.	service, dao, ml, exception
com.locus.service.validation	Input validators.	model.dto, exception
com.locus.ml	LinearRegressionPredictor (consumes ml/model.json)	(stdlib only)
com.locus.dao	DAO interfaces.	model, model.dto, exception
com.locus.dao.impl	JDBC implementations using HikariCP.	dao, config
com.locus.config	DBConnection (HikariCP DataSource).	(stdlib + HikariCP)
com.locus.model (+ .dto)	POJOs and DTOs.	(none)
com.locus.exception	ValidationException, DataAccessException.	(none)

External Python packages — **etl/** (run_etl.py, zameen_scraper.py) and **ml/** (train_model.py, retrain_model.py) — sit outside the Java module graph and communicate solely through the PostgreSQL database and the *ml/model.json* artefact. The **data/** package (generate_seed.py) is invoked once at setup.

9. Deployment Diagram

LOCUS Analytics is deployed on a single workstation (the analyst's machine) plus a PostgreSQL database that may be local or networked. The deployment view is summarised below; a detailed PNG rendering of the schema (foreign keys and indexes) is referenced from the assets folder.

Node	Artefact	Notes
Analyst Workstation (Win/Linux/macOS)	locus-analytics-1.0-SNAPSHOT.jar (fat JAR)	JavaFX 21 client, JDK 17 runtime
Analyst Workstation	Python 3.10 + run_etl.py / train_model.py / retrain_model.py	invokes ETL ServiceImpl as subprocess
Analyst Workstation (filesystem)	ml/model.json, generated-reports/, backups/ hotfiledumps/	model; PDF outputs; weekly DB backups
Database Server	PostgreSQL 15+ (database: locus_analytics)	schema.sql provisions tables, indexes, audit triggers
External Service	Google Maps JavaScript API	consumed by the Heatmap WebView only

Communication links: (1) JavaFX client ↔ PostgreSQL via JDBC (TCP/5432); (2) JavaFX client ↔ Python ETL/ML via OS process spawn + the shared *etl_job* table; (3) JavaFX WebView ↔ Google Maps over HTTPS; (4) Backup scripts ↔ filesystem (*backups/*) on a weekly cron/Task Scheduler trigger.

Build & release: a single *mvn clean package* produces the shaded JAR; deployment is a copy-and-run, with *config.properties* as the only host-specific file. Database upgrades are applied via versioned SQL scripts (*schema.sql* and *fix_bad_data.sql*).